Universidad Nacional Rosario Castellanos

Licenciatura en Ciencia de Datos para Negocios

Unidad Académica Chalco · Tercer semestre · Semestre 2026-2

UCA: Estructuras de Datos (LCDN3EST8)

Problema prototípico: Bacheo inteligente y priorización urbana

SIPA

Sistema Inteligente de Priorización Asfáltica

Avance del problema prototípico e incidente crítico

#	Integrantes del equipo
1	**Sandra Luna Altos**
2	Joshua David Aquino Soto
3	Rebeca Jaimes Cruz
4	Ithan Felipe Santiago

Grupo	Docente	Fecha de entrega
302	**Luisa Reyna Beatriz Ayon Muñoz**	**02 Octubre de 2026**

Contenido

Contenido 2

Introducción 3

Parte I. Avance del problema prototípico 3

1. Problemática identificada 3

2. Solución propuesta: SIPA 4

3. Definición de los datos 5

Parte II. Incidente crítico: Estructuras de Datos 8

5. Síntesis del incidente 8

6. Planteamiento de preguntas 8

7. Aplicación de los temas de la UCA en SIPA 9

8. Modelado con programación orientada a objetos 10

9. Estructuras y algoritmos seleccionados 10

10. Diagramas de flujo del programa 13

11. Pseudocódigo de los procesos principales 16

12. Cómo comprobaremos la eficiencia y el impacto 17

13. Reflexión ética y próximos pasos 18

Referencias 18

Introducción

Este documento reúne los avances del equipo en el problema prototípico del tercer semestre, «Bacheo inteligente y priorización urbana», y en el incidente crítico de la UCA Estructuras de Datos. Está organizado en dos partes:

- **Parte I. Problema prototípico:** la problemática identificada, la solución que elegimos (SIPA), la definición de los datos con los que trabajaremos, la limpieza que ya realizamos y el estado de avance.
- **Parte II. Incidente crítico de Estructuras de Datos:** las preguntas que nos planteamos, el modelado con programación orientada a objetos, las estructuras y algoritmos que vamos a implementar, los diagramas de flujo del programa, el pseudocódigo de los procesos principales y la forma en que comprobaremos su eficiencia.

Parte I. Avance del problema prototípico

1. Problemática identificada

El deterioro de las vialidades afecta la movilidad, la seguridad vial, los costos de transporte y la calidad de vida. La magnitud del problema se refleja en el gasto público: en 2025 la Ciudad de México destinó 2,250 millones de pesos, 50 brigadas y 480 trabajadores a un Programa Integral de Bacheo Nocturno para intervenir 1,250 km de 217 vialidades primarias (El País, 2025a).

Al analizar el problema prototípico identificamos que la dificultad principal no es saber **dónde** hay baches, sino decidir **cuáles atender primero y por qué**. A partir de la lectura y de la revisión de los datos encontramos cuatro problemas concretos:

Problema	Qué observamos	Consecuencia
Atención reactiva y por orden de llegada (FIFO)	**Los reportes se atienden en el orden en que se registran o según la presión social.**	**Baches graves en avenidas con transporte público pueden esperar detrás de baches menores reportados antes.**
El número de reportes no mide el impacto	Una vialidad con pocos reportes puede ser usada por miles de personas al día.	Se subestima el riesgo de vías importantes y se sobrestima el de calles con mucha participación.
Sesgo de participación digital	En la base de la CDMX, Iztapalapa (la alcaldía más poblada) registra 1,685 solicitudes, frente a 4,210 de Álvaro Obregón.	Si solo se cuentan reportes, las zonas con menos acceso o participación quedan rezagadas.

Problema	Qué observamos	Consecuencia
Reportes duplicados y datos dispersos	Varios vecinos reportan el mismo bache; la información de tránsito, accidentes y rutas está en fuentes separadas.	Se saturan las colas de atención y no se puede comparar el riesgo real entre vialidades.

Por ello, adoptamos la pregunta que propone el problema prototípico: dejar de preguntar «¿qué bache se reportó primero?» y preguntar **«¿qué intervención debe realizarse primero para reducir el mayor riesgo y generar el mayor beneficio social con los recursos disponibles?»**.

2. Solución propuesta: SIPA

Nuestra propuesta es **SIPA (Sistema Inteligente de Priorización Asfáltica)**, una plataforma que registra y depura los reportes, estima el riesgo de cada tramo, calcula un **Índice de Prioridad Urbana (IPU)** con varios criterios, lo muestra en un semáforo (verde, amarillo y rojo) y propone qué reparar primero y por qué ruta, dejando la decisión final a una persona. Funciona en cuatro pasos:

Paso	Qué hace	Pregunta que responde
1. Predice	**Estima la probabilidad de baches nuevos, el riesgo de accidente a 72 horas y la degradación del asfalto.**	**¿Dónde es más probable que haya problemas?**
2. Prioriza	Combina los criterios en el IPU y clasifica cada tramo con el semáforo.	¿Qué se debe atender primero?
3. Optimiza	Selecciona lo que cabe en el presupuesto y traza rutas para las brigadas.	¿Cómo aprovechar mejor los recursos?
4. Justifica	Genera una ficha con los datos de cada decisión y registra los ajustes humanos.	¿Por qué esta intervención y no otra?

El IPU es una suma ponderada de criterios normalizados entre 0 y 1. Los pesos siguientes son una propuesta inicial que justificaremos en el plan de viabilidad:

Criterio del IPU	Fuente o cálculo	Peso inicial
Nivel de deterioro	**Profundidad, área y severidad del bache**	**0.20**
Riesgo de accidente a 72 h	Modelo de probabilidad con historial de accidentes	0.20
Personas afectadas	Aforo × ocupación + pasajeros de transporte público	0.15
Relevancia para el transporte público	Número e importancia de rutas en el tramo	0.10
Intensidad del tránsito	Aforo y porcentaje de carga pesada	0.10
Vulnerabilidad territorial	Índice de marginación de la zona	0.10
Riesgo de deterioro mayor	Probabilidad de baches nuevos y degradación acumulada	0.10

Criterio del IPU	Fuente o cálculo	Peso inicial
Reportes ciudadanos (acotado)	Escala logarítmica con tope, para evitar sesgos	0.05

3. Definición de los datos

En lugar de levantar una encuesta como fuente principal, decidimos trabajar con **datos reales ya existentes**, porque permiten calibrar y validar los modelos con miles de registros. Los criterios que no están en esas bases se completarán con fuentes oficiales o con datos sintéticos, indicándolo siempre.

3.1 Bases de datos disponibles

Característica	base_maestra_baches_2019_2024.csv	baches_dataset.csv
Contenido	**Solicitudes ciudadanas de bacheo (un reporte por fila)**	**Eventos tomados de noticias: programas de bacheo, socavones, quejas, accidentes**
Territorio	Ciudad de México (16 alcaldías)	Principalmente Estado de México (127 registros de Chalco y Valle de Chalco)
Periodo real	Junio de 2019 a diciembre de 2022	Principalmente 2025 y 2026
Registros	35,743	4,585
Columnas	Folio, fecha, colonia, alcaldía, estatus, latitud, longitud	Fecha, texto del evento, municipio, colonia, calle, CP, latitud, longitud, nivel y radio de geolocalización
Uso en SIPA	Núcleo cuantitativo: alimentar el prototipo y calibrar los modelos	Evidencia cualitativa: casos de alto impacto, argumentación y contexto de Chalco

Decisión del equipo

Las dos bases no se combinan en un mismo cálculo porque no comparten territorio ni periodo. La base de noticias tampoco se usará para calcular tasas, porque su crecimiento refleja la recolección de notas y no un aumento real de baches.

3.2 Variables que usará el prototipo

La unidad que se prioriza es el **bache** (agrupando los reportes que se refieren a él) dentro de un **tramo** o celda geográfica. Las variables principales son:

Variable	Tipo	Origen	Uso
id_folio	**Entero**	**Base maestra**	**Identificador del reporte**
fecha_solicitud	Fecha	Base maestra	Orden de llegada, estacionalidad, tiempo de espera

Variable	Tipo	Origen	Uso
latitud, longitud	Decimal	Base maestra	Clave de la tabla hash, mapa, nodos del grafo
alcaldía, colonia	Texto	Base maestra (completada por vecinos)	Agrupar por zona y medir equidad
estatus	Catálogo	Base maestra (agrupado)	Resuelto, turnado o pendiente
celda_id (250 m)	Texto	Calculada	Unidad de análisis («tramo»)
severidad (1 a 5)	Entero	Sintética, según la norma SICT de deterioros	Criterio de deterioro del IPU
aforo y % carga pesada	Entero / decimal	Referencia o sintética por tipo de vía	Tránsito y degradación
rutas de transporte	Lista	Datos abiertos CDMX (GTFS)	Relevancia para el transporte público
accidentes cercanos	Entero	INEGI, ATUS	Riesgo de accidente a 72 h
índice de marginación	Decimal	CONAPO	Vulnerabilidad territorial
costo estimado	Decimal	Modelo de degradación (Cálculo Integral)	Selección con presupuesto

3.3 Limpieza de la base maestra

Ya programamos un script en Python (scripts/limpiar_base_maestra.py) que limpia la base sin borrar ni modificar el archivo original. Todo lo que se corrige o se infiere queda marcado en columnas de control, y cada paso se registra en una bitácora. Los resultados fueron:

Paso de limpieza	Registros
Registros leídos	**35,743**
Prefijo «- » eliminado del folio	35,743
Texto normalizado en colonia y alcaldía (espacios, mayúsculas)	605
Excluidos por coordenadas fuera de la CDMX (por ejemplo, en otros países)	114
Estatus agrupados con un catálogo (ATENDIDO y CERRADO → RESUELTO)	30,732
Alcaldía inferida a partir de reportes vecinos (≤ 1,000 m)	109
Colonia inferida a partir de reportes vecinos (≤ 150 m)	158
Marcados como ubicación dudosa (sin alcaldía)	47

Paso de limpieza	Registros
Reportes que comparten ubicación (≈ 11 m) con otro reporte	921
Marcados como posible duplicado (misma celda de ≈ 11 m y menos de 30 días)	158
Registros en periodos atípicos (pandemia 2020 y hueco dic. 2021–ene. 2022), marcados	684
Celdas de 250 m con al menos un reporte	8,323
Registros finales en la base limpia	**35,629**

Los datos de duplicados (921 y 158) son importantes para la Parte II, porque nos sirven para calibrar la tolerancia de la tabla hash que eliminará reportes repetidos.

3.4 Hallazgos iniciales

- **Estacionalidad:** las solicitudes aumentan en temporada de lluvias y alcanzan su máximo en septiembre (1,402 en promedio, frente a 306 en diciembre). Esto justifica incluir un factor de temporada en el modelo de probabilidad.
- **Puntos críticos:** al agrupar en celdas de ≈ 110 m, 8,205 celdas tienen más de un reporte y la más recurrente acumula 30.
- **Sesgo de participación:** la diferencia entre Iztapalapa y Álvaro Obregón no prueba que haya menos baches, pero muestra que contar reportes no basta. Por eso los reportes tienen un peso acotado (0.05) en el IPU.
- **Limitaciones:** la base cuenta reportes, no baches, y no trae severidad, calle ni fecha de cierre. Esos datos se completarán con fuentes externas o datos sintéticos.

3.5 Instrumento complementario: encuesta de percepción (propuesta)

El plan de viabilidad pide contrastar la percepción inicial con la evidencia. Para documentar esa percepción proponemos una encuesta breve dirigida a estudiantes y vecinos de Chalco, que se aplicaría antes de mostrar los resultados del modelo:

1. ¿Por qué calles o avenidas transitas con más frecuencia y en qué medio de transporte?
2. ¿Cuáles consideras que son las tres vialidades con baches más urgentes de reparar en tu zona?
3. ¿Por qué las consideras urgentes? (riesgo de accidente, tráfico, transporte público, daño a vehículos, otro)
4. ¿Has reportado algún bache? ¿Por qué medio? Si no, ¿por qué no?
5. ¿Has sufrido algún daño o accidente por un bache en el último año?
6. ¿Qué debería pesar más al decidir qué reparar primero: el número de reportes, el riesgo o la cantidad de personas afectadas?

Las respuestas se guardarán sin nombre ni teléfono, siguiendo el principio de minimización de datos personales.

Parte II. Incidente crítico: Estructuras de Datos

4. Síntesis del incidente

El incidente plantea que el reto no está solo en recibir reportes, sino en la **eficiencia con la que el sistema los procesa, filtra, organiza y prioriza**. Un sistema que atiende en orden de llegada (FIFO) genera asignaciones reactivas, recorridos desordenados de las brigadas y rezago de daños graves en avenidas con transporte público. Además, la llegada masiva de reportes repetidos sobre un mismo bache satura las colas de procesamiento.

El incidente propone tres retos técnicos: (1) gestionar los reportes con estructuras dinámicas y una **cola de prioridad (max-heap)** ordenada por el IPU; (2) **eliminar duplicados** con tablas hash o árboles AVL; y (3) calcular **rutas óptimas** representando la red vial como un **grafo** y aplicando Dijkstra. Todo ello sin generar sesgos que dejen desatendidas ciertas zonas de la ciudad.

> **Dilema del incidente**
>
> **¿Cómo diseñar una arquitectura de estructuras de datos que procese eficientemente grandes volúmenes de información, optimizando tiempo y memoria, sin generar sesgos que dejen desatendidas a ciertas zonas y garantizando una asignación equitativa, transparente y óptima de los recursos públicos?**

5. Planteamiento de preguntas

A partir del incidente, el equipo se planteó las siguientes preguntas. La columna derecha resume nuestra respuesta inicial, que se desarrolla en las secciones siguientes:

#	Pregunta	Respuesta inicial del equipo
1	**¿Por qué una cola FIFO no es suficiente para atender baches?**	**Porque ordena por tiempo de llegada y no por riesgo. Un bache grave reportado tarde queda detrás de baches menores.**
2	¿Qué estructura mantiene siempre al frente el bache con mayor IPU y a qué costo?	Un max-heap: insertar y extraer el máximo cuesta O(log n), y consultar el máximo O(1).
3	¿Cómo saber rápido si un reporte nuevo es de un bache ya registrado?	Con una tabla hash cuya clave es la celda geográfica (≈ 11 m): búsqueda O(1) en promedio, en lugar de recorrer todos los reportes (O(n)).
4	¿Tabla hash o árbol AVL?	Hash como estructura principal por su rapidez; AVL como comparación, porque garantiza O(log n) en el peor caso y permite recorrer las claves en orden.
5	¿A qué distancia dos reportes son el mismo bache?	≈ 11 m y menos de 30 días, calibrado con la base real (921 reportes comparten ubicación; 158 cumplen ambas condiciones).
6	¿Cómo cambiar la prioridad de un bache que ya está en el heap?	Guardando su posición en el arreglo del heap (en un diccionario) para subirlo o bajarlo en O(log n) sin reconstruir todo.

#	Pregunta	Respuesta inicial del equipo
7	¿Cómo representar la red vial y calcular rutas?	Grafo ponderado con lista de adyacencia: intersecciones como vértices y calles como aristas con distancia o tiempo. Dijkstra con nuestro propio heap.
8	¿Cómo evitar que zonas con baja prioridad nunca se atiendan?	Con un factor de envejecimiento acotado que sube poco a poco la prioridad de lo que lleva mucho tiempo en espera, y midiendo tiempos por zona.
9	¿Cómo permitir que una persona corrija y deshaga decisiones?	Con una pila de operaciones (deshacer en O(1)) y una bitácora que exige el motivo de cada ajuste.
10	¿Cómo demostrar que las estructuras cumplen las complejidades?	Midiendo tiempos con 1,000, 10,000 y 35,629 registros reales y comparando con la versión lineal.

6. Aplicación de los temas de la UCA en SIPA

Tema de la UCA	Aplicación en SIPA
A. Programación orientada a objetos	**Clases Reporte, Bache, Tramo, Brigada y GrafoVial, con sus atributos y métodos (sección 8).**
B. Pilas	Historial de decisiones del supervisor para deshacer y rehacer; recorrido en profundidad (DFS) para verificar que el grafo vial esté conectado.
C. Recursividad	Inserción, rotaciones y recorrido en orden del árbol AVL; DFS recursivo sobre el grafo.
D. Conjuntos	Celdas ya visitadas en Dijkstra, zonas y colonias sin repetir, baches asignados a cada brigada.
E. Colas	Cola FIFO como línea base para comparar; cola de prioridad (max-heap) para el orden de atención.
F. Listas enlazadas	Lista doblemente enlazada de reportes entrantes: insertar, cancelar o fusionar reportes en O(1) sin reacomodar arreglos. Listas encadenadas en las cubetas de la tabla hash.
G. Árboles	Árbol AVL indexado por celda geográfica para comparar contra la tabla hash; el heap es un árbol binario completo guardado en un arreglo.
H. Grafos	Red vial como grafo ponderado; Dijkstra para rutas mínimas desde la planta de asfalto; vecino más cercano para el recorrido de la brigada.

7. Modelado con programación orientada a objetos

Clase	Atributos principales	Métodos principales
Reporte	**id_folio, fecha, latitud, longitud, fuente, foto (opcional), anterior, siguiente**	**clave_celda(), cancelar()**

Clase	Atributos principales	Métodos principales
Bache	id, clave_celda, severidad, num_reportes, fecha_primer_reporte, estado, ipu, costo	agregar_reporte(), calcular_ipu(pesos), dias_en_espera()
Tramo	celda_id, alcaldía, colonia, aforo, rutas_transporte, marginación, accidentes	criterios_normalizados()
ListaReportes	cabeza, cola, tamaño	insertar_final(), eliminar(nodo), fusionar(otra)
TablaHash	cubetas (listas), m, n	buscar(clave), insertar(clave, bache), rehash()
ArbolAVL	raíz	insertar(clave), buscar(clave), rotar_izq(), rotar_der(), en_orden()
MaxHeap	arreglo, posición (diccionario id → índice)	insertar(), extraer_max(), actualizar(id, prioridad), subir(), bajar()
GrafoVial	adyacencia (diccionario de listas)	agregar_arista(), dijkstra(origen), dfs(origen)
Brigada	id, base, capacidad_diaria, ruta	asignar(baches), recorrido_vecino_cercano()
PilaDecisiones	elementos	apilar(decisión), deshacer()

Regla de implementación

Las estructuras se programarán desde cero en Python, sin módulos que ya las resuelven (por ejemplo, heapq), para poder demostrar el cumplimiento de las complejidades que se evaluarán en la UCA.

8. Estructuras y algoritmos seleccionados

8.1 Entrada de reportes: lista doblemente enlazada

Los reportes que llegan se guardan en una lista doblemente enlazada. Cada nodo apunta al anterior y al siguiente, por lo que insertar al final, cancelar un reporte o fusionarlo con otro cuesta O(1) si se tiene la referencia al nodo. Con un arreglo, eliminar un elemento intermedio obliga a recorrer los elementos siguientes (O(n)).

8.2 Eliminación de duplicados: tabla hash (y AVL como comparación)

La latitud y la longitud se redondean a cuatro decimales (≈ 11 m) y forman la **clave** de la tabla hash. Si la clave ya existe y el último reporte de ese bache tiene menos de 30 días, el reporte se suma al bache existente; si no, se registra como reincidencia o como bache nuevo. Las colisiones se resuelven por **encadenamiento** (cada cubeta es una lista) y la tabla se duplica cuando el factor de carga supera 0.75.

Para no separar dos reportes del mismo bache que caen justo en el borde de dos celdas, también se revisan las 8 celdas vecinas. Son 9 búsquedas de O(1), así que el costo sigue siendo constante.

¿Por qué no una búsqueda lineal? Con los 35,629 registros de la base limpia, comparar cada reporte nuevo contra todos los anteriores requiere del orden de $n^2/2 \approx 634$ millones de comparaciones. Con la

tabla hash son alrededor de 35,629 × 9 ≈ 320 mil búsquedas. Con el árbol AVL, cada búsqueda toma a lo más ≈ $1.44 \cdot \log_2(n) \approx 22$ comparaciones.

Criterio	Tabla hash	Árbol AVL
Buscar / insertar	**O(1) promedio, O(n) peor caso**	**O(log n) garantizado**
Recorrer claves en orden	No	Sí (recorrido en orden)
Búsqueda por rango (zona)	No directamente	Sí
Implementación	Más sencilla	Requiere rotaciones (recursividad)
Uso en SIPA	Estructura principal de desduplicación	Comparación de rendimiento y consultas por rango

8.3 Orden de atención: cola de prioridad (max-heap)

Cada bache entra a un max-heap con su IPU como prioridad. El heap se guarda en un arreglo: los hijos del nodo i están en 2i+1 y 2i+2. Al insertar, el elemento «sube» mientras sea mayor que su padre; al extraer el máximo, el último elemento pasa a la raíz y «baja». Ambas operaciones cuestan O(log n). Como el IPU de un bache cambia cuando llegan nuevos reportes, guardamos en un diccionario la posición de cada bache dentro del arreglo para actualizar su prioridad en O(log n).

El siguiente ejemplo ilustrativo (IPU supuestos) muestra la diferencia entre atender por orden de llegada y atender por prioridad:

Reporte	Hora	Descripción	IPU	Semáforo	FIFO	Heap
R1	**08:05**	**Calle residencial cerrada, bache superficial**	**0.31**	**Verde**	**1.º**	**5.º**
R2	08:20	Avenida con 3 rutas de transporte, bache profundo	0.84	Rojo	2.º	2.º
R3	08:40	Calle secundaria, bache mediano	0.52	Amarillo	3.º	4.º
R4	09:10	Vía rápida con carga pesada, bache profundo	0.91	Rojo	4.º	1.º
R5	09:30	Acceso a escuela en zona de alta marginación	0.67	Amarillo	5.º	3.º

Con FIFO, el bache de mayor riesgo (R4) se atiende en cuarto lugar; con el heap, en primero.

8.4 Equidad: envejecimiento acotado de la prioridad

Un riesgo de cualquier cola de prioridad es la **inanición**: los elementos de baja prioridad podrían no atenderse nunca. Para responder al dilema del incidente proponemos una prioridad efectiva:

prioridad = IPU + β · mín(días_en_espera, D) / D, con β = 0.10 y D = 60 días (valores por calibrar).

El aumento máximo es de 0.10, así que un bache verde que lleva mucho tiempo esperando sube de lugar, pero nunca supera a un bache rojo. Además, la vulnerabilidad territorial forma parte del IPU y el tablero mostrará el tiempo de espera promedio por zona para detectar si alguna se queda rezagada.

8.5 Plan diario y rutas: selección con presupuesto, grafo y Dijkstra

Para armar el plan del día se extraen del heap los baches de mayor prioridad mientras haya presupuesto y capacidad de brigadas; los que no caben se guardan y se reinsertan al final. Después se calculan las rutas:

- **Grafo vial:** lista de adyacencia donde los vértices son intersecciones (o centros de celda, mientras no se integre la red vial de OpenStreetMap o del INEGI) y las aristas son calles con peso igual a la distancia o al tiempo de traslado.
- **Dijkstra:** desde la planta de asfalto calcula la distancia mínima a cada bache seleccionado, usando nuestro propio heap (de mínimos). Complejidad O((V + E) log V).
- **Recorrido de la brigada:** con las distancias de Dijkstra se arma el recorrido con la heurística del vecino más cercano (ir siempre al bache más cercano no visitado). No garantiza la ruta perfecta, pero evita recorridos desordenados.

8.6 Resumen de complejidades

Estructura	Operación	Complejidad esperada
Lista doblemente enlazada	**Insertar al final / eliminar un nodo conocido**	**O(1)**
Tabla hash	Buscar o insertar una celda	O(1) promedio
Árbol AVL	Buscar o insertar una celda	O(log n)
Max-heap	Insertar, extraer máximo, actualizar prioridad	O(log n)
Grafo + Dijkstra	Camino mínimo desde un origen	O((V + E) log V)
Pila	Apilar / deshacer	O(1)
Conjunto	Verificar pertenencia	O(1) promedio

9. Diagramas de flujo del programa

La Figura 1 muestra el flujo general de SIPA desde que llega un reporte hasta que se generan las órdenes de trabajo. Las Figuras 2 y 3 detallan los dos procesos centrales del incidente: la eliminación de duplicados y la construcción del plan diario.

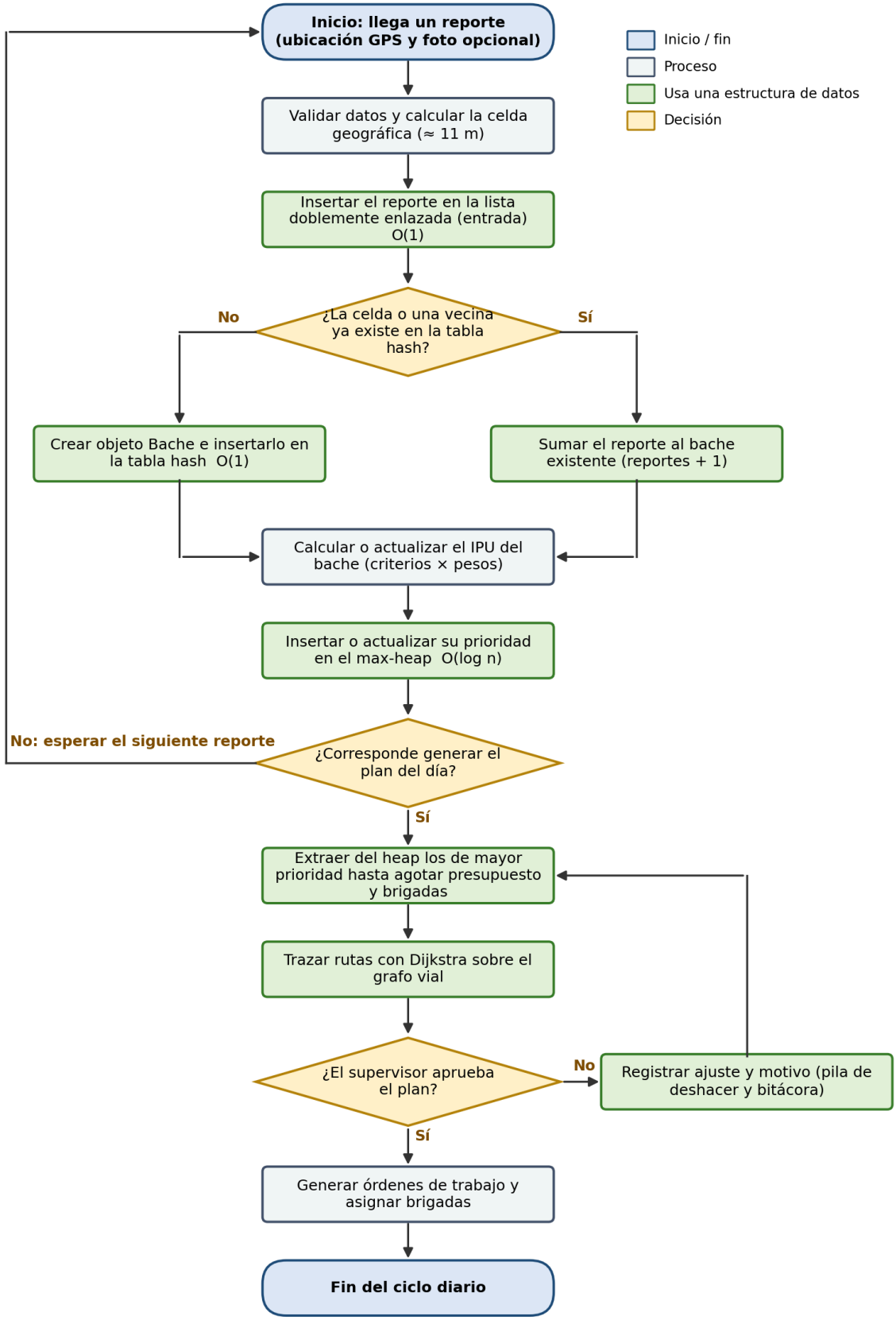


Figura 1. Flujo general del procesamiento de reportes en SIPA.

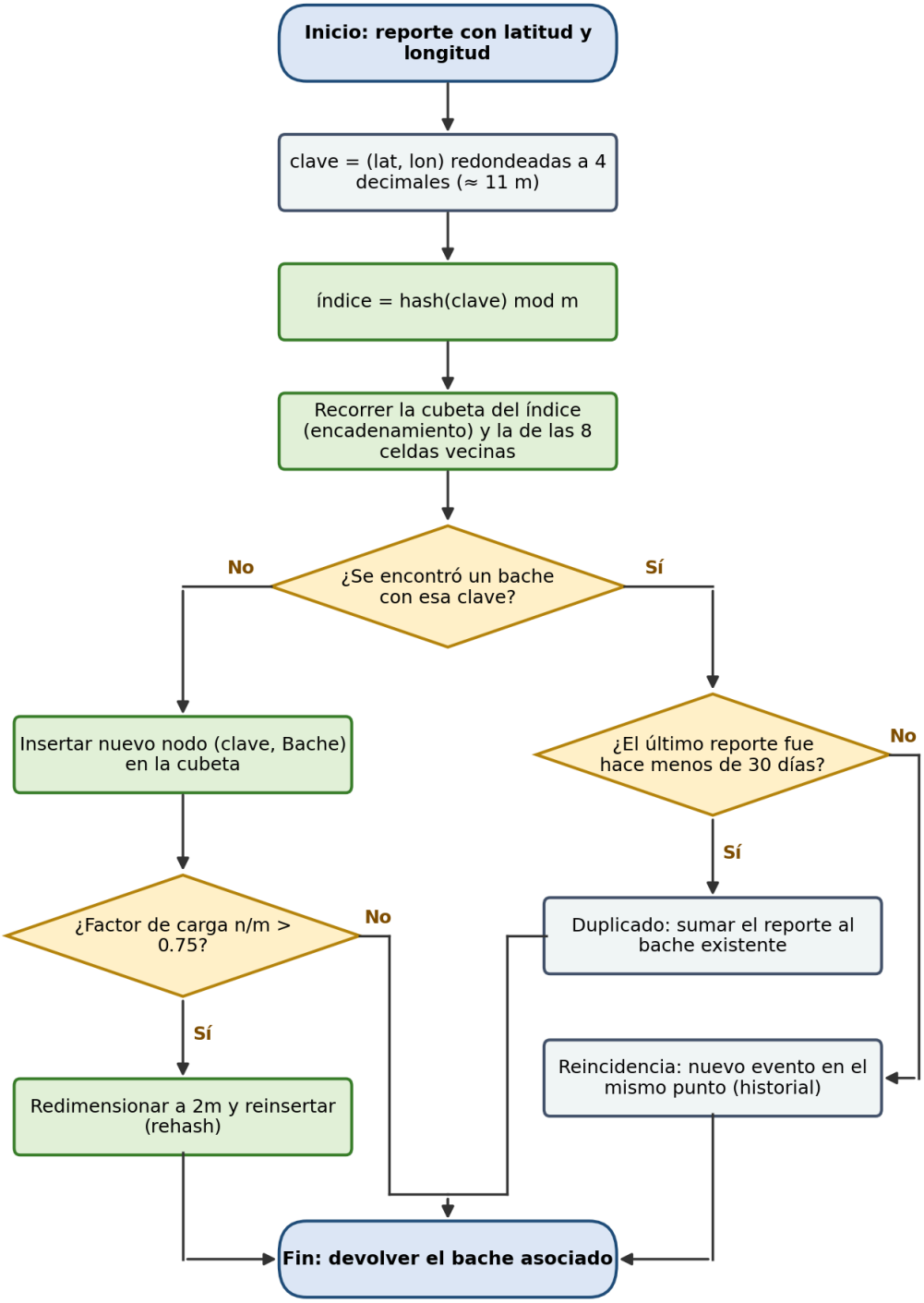


Figura 2. Eliminación de duplicados con tabla hash.

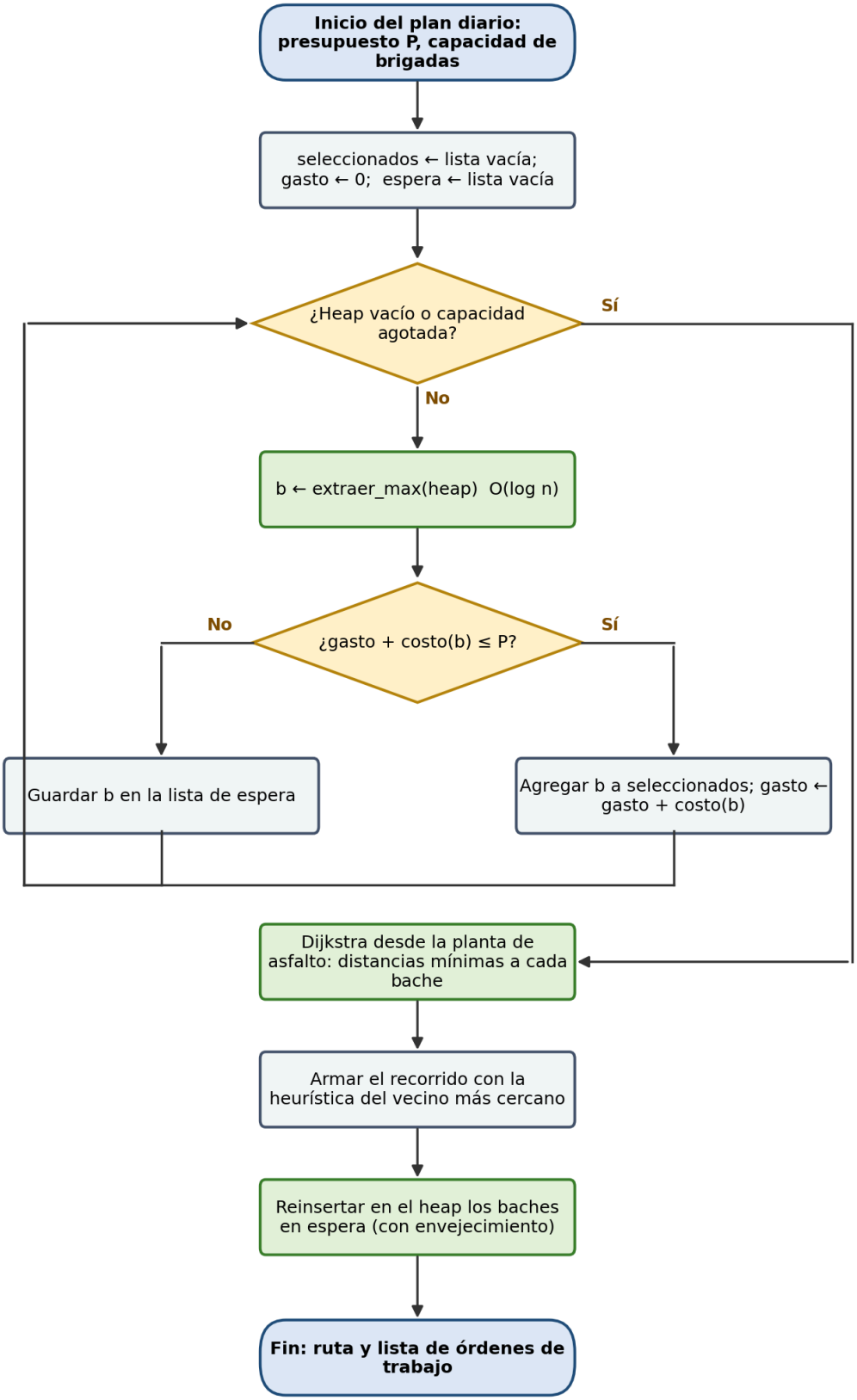


Figura 3. Plan diario: selección con presupuesto y cálculo de rutas.

10. Pseudocódigo de los procesos principales

10.1 Registrar un reporte

```
función registrar_reporte(reporte):
    lista_reportes.insertar_final(reporte) # O(1)
    clave ← (redondear(reporte.lat, 4), redondear(reporte.lon, 4))
    bache ← nulo
    para cada clave_v en clave y sus 8 celdas vecinas: # 9 búsquedas O(1)
        bache ← tabla_hash.buscar(clave_v)
        si bache ≠ nulo: salir del ciclo
    si bache = nulo:
        bache ← nuevo Bache(clave, reporte)
        tabla_hash.insertar(clave, bache)
    si no, si reporte.fecha − bache.ultimo_reporte < 30 días:
        bache.agregar_reporte(reporte) # duplicado
    si no:
        bache.registrar_reincidencia(reporte)
    bache.ipu ← bache.calcular_ipu(pesos_del_territorio)
    heap.insertar_o_actualizar(bache.id, prioridad(bache)) # O(log n)
```

10.2 Max-heap: insertar y extraer el máximo

```
función insertar(elemento):
    arreglo.agregar(elemento); i ← tamaño − 1
    subir(i)
función subir(i):
    mientras i > 0 y arreglo[padre(i)].prioridad < arreglo[i].prioridad:
        intercambiar(i, padre(i)) # también actualiza el diccionario posición
        i ← padre(i) # padre(i) = (i − 1) // 2
función extraer_max():
    máximo ← arreglo[0]
    arreglo[0] ← arreglo[último]; eliminar último
    bajar(0) # intercambia con el hijo mayor (2i+1, 2i+2)
    regresar máximo
```

10.3 Dijkstra sobre el grafo vial

```
función dijkstra(grafo, origen):
  dist ← diccionario con ∞ para cada vértice; dist[origen] ← 0
  visitados ← conjunto vacío
  heap_min.insertar(origen, 0)
  mientras heap_min no esté vacío:
    u ← heap_min.extraer_min()
    si u está en visitados: continuar
    visitados.agregar(u)
    para cada (v, peso) en grafo.adyacencia[u]:
      si dist[u] + peso < dist[v]:
        dist[v] ← dist[u] + peso
        heap_min.insertar(v, dist[v])
  regresar dist
```

11. Cómo comprobaremos la eficiencia y el impacto

1. **Pruebas de complejidad:** medir el tiempo de inserción y búsqueda con 1,000, 10,000 y 35,629 registros reales de la base limpia, comparando búsqueda lineal, tabla hash y árbol AVL. Si el tiempo de la tabla hash se mantiene casi constante y el del AVL crece de forma logarítmica, se comprueban las complejidades.
2. **Pruebas de correctitud:** verificar que el heap siempre extrae el mayor IPU, que el AVL queda balanceado después de cada inserción y que Dijkstra coincide con cálculos hechos a mano en un grafo pequeño.
3. **Simulación FIFO vs. SIPA:** generar 30 días de reportes, fijar una capacidad (por ejemplo, 5 brigadas) y atenderlos con ambas estrategias. Se compararán el tiempo de espera de los baches rojos, los kilómetros recorridos y el tiempo de espera promedio por zona (equidad).

12. Reflexión ética y próximos pasos

El incidente nos hizo ver que elegir una estructura de datos no es solo una decisión técnica. Una cola FIFO parece justa porque respeta el turno, pero ignora el riesgo; un heap que solo usa el IPU es eficiente, pero puede dejar zonas sin atender si no se controla. Por eso SIPA combina criterios de riesgo y vulnerabilidad, limita el peso de los reportes, aplica envejecimiento acotado, mide la equidad por zona y deja la decisión final a una persona que debe justificar sus cambios.

Próximos pasos del equipo:

- Programar y probar la lista doblemente enlazada, la tabla hash, el AVL, el max-heap, la pila y el grafo con Dijkstra.
- Conectar las estructuras con la base limpia (35,629 registros) y medir sus tiempos.
- Construir un grafo vial de prueba para una zona pequeña y calcular rutas.
- Aplicar la encuesta de percepción antes de mostrar resultados del modelo.

- Integrar el módulo de estructuras con la base de datos en SQL Server y con el tablero.

Referencias

Bel, W. (2020). Algoritmos y estructuras de datos en Python. Un enfoque ágil y estructurado. Editorial UADER.

Cámara de Diputados del H. Congreso de la Unión. (2025). Ley General de Protección de Datos Personales en Posesión de Sujetos Obligados. Diario Oficial de la Federación.

El País. (2025a, 21 de agosto). Programa Integral de Bacheo Nocturno: horarios y avenidas que estarán cerradas en la primera etapa. El País México.

El País. (2025b, 20 de septiembre). Los socavones de Iztapalapa: una herida que no se cierra en Ciudad de México. El País México.

Gobierno de la Ciudad de México. (2025, 20 de agosto). Arrancamos con el Programa Integral de Bacheo Nocturno en la Ciudad de México. Jefatura de Gobierno. https://gobierno.cdmx.gob.mx/noticias/arrancamos-con-el-programa-integral-de-bacheo-nocturno-en-la-ciudad-de-mexico/

Instituto Nacional de Estadística y Geografía. (2024). Estadística de accidentes de tránsito terrestre en zonas urbanas y suburbanas (ATUS) 2024. INEGI. https://www.inegi.org.mx/programas/accidentes/

Mohan, A., Sarma, K. V., et al. (2022). Road network deterioration monitoring using aerial images and computer vision. arXiv. https://arxiv.org/abs/2209.15455

Python Software Foundation. (2023). The Python Tutorial: Data Structures. https://docs.python.org/3/tutorial/datastructures.html

Saini, S., et al. (2025). iWatchRoadv2: Pothole detection, geospatial mapping, and intelligent road governance. arXiv. https://arxiv.org/abs/2510.16375

Secretaría de Infraestructura, Comunicaciones y Transportes. (2023). Determinación de los deterioros superficiales de los pavimentos (M·CSV·CAR·1·03·008/23). SICT.